Historical Confidentiality for Encrypted Agent Memory: Dual-Epoch Security and the Channels That Defeat It

Shreyas Fegade

Abstract—AI agents write their own long-term memory to an encrypted store and retrieve it by semantic search. We ask what an adversary who obtains that storage midway, through a synchronized backup or repeated disk images, learns about records written *before* the compromise. This historical confidentiality is not provided by encryption, snapshot security or history independence. In stores that place records by content, topic labels the adversary attaches to partitions by inducing new writes transfer to earlier records whenever the layout discloses their partition; pgvector under an encrypted file system does. Sealing the layout is not enough: retained versions name an earlier record's region at 13 times the chance rate, capacity refusals expose a topic's load, and the agent itself leaks. A pipeline that skips facts it already holds turns the occurrence of a write into a membership oracle, with AUC 0.83 in the unmodified mem0 library whenever its model restates a stored fact verbatim. We define dual-epoch security as an adaptive, leakage-parameterized game, prove sufficient conditions that tolerate overflow, separate it from breach resistance and history independence, and prove that against a self-writing agent a store is secure exactly when what it reveals is independent of what the agent does. A store with content-independent placement, search in locked memory and a clocked write-only-oblivious sweep meets that condition: placed under unmodified mem0 it takes the oracle to chance (AUC 0.47), it writes 68.4 KiB per flush, and it extends past the memory limit with its writes still hidden.

Index Terms—Searchable encryption, encrypted vector search, leakage-abuse attacks, history independence, snapshot adversaries, AI agent memory.

1 Introduction

LARGE language model agents increasingly keep a persistent memory. While triaging tickets, reading documents, or debugging code, an agent records what it has learned and later retrieves relevant records by nearest-neighbor search over text embeddings [1]. The records are written continuously and without human review, and the memory is often stored on the user's own device. Over time it becomes a detailed account of the work the agent has performed.

Security research on agent memory has so far concentrated on integrity. Poisoning attacks such as MINJA [2], AgentPoison [3], and MemoryGraft [4] insert records that alter the agent's later behavior, and the OWASP LLM Top 10 lists vector and embedding weaknesses as a distinct risk [5]. Confidentiality has received less attention. MEXTRA [6] extracts memory records through the agent's own prompting interface, and membership inference against chat-agent memory likewise queries the running agent [7], but no existing work models an adversary that obtains the storage layer. Encrypting the memory is the natural first measure, because embeddings can be inverted to recover much of the original text [8].

Consider a coding agent that spent the spring helping its user prepare a confidential acquisition. Its memory folder is encrypted and synchronized to a cloud account, and in June that account is phished. From then on the attacker receives every new version of the store, and it can email the agent documents that the agent will read and remember. It cannot decrypt anything. What does it learn about March?

This paper studies that adversary, which obtains the storage layer at a time τ through a synchronization or backup account or repeated disk imaging, and holds only stored state for the records written before τ. Those records might be expected to be safer for having been written before the breach. Our thesis is that this expectation is a security property in its own right — *historical confidentiality*, the analogue for stored structure of what forward secrecy is for keys — and that encryption, snapshot security and history independence do not deliver it. Figure 1 summarizes the four channels by which an adversary reaches the earlier records, and the design that closes them.

The attack, and its precondition: The attack rests on two properties of the setting. An agent's writes can be induced: an adversary that supplies a document, a ticket, or an email causes the agent to write a record about it, and that record is ordinary agent output. And encrypted vector indexes partition their records by content, so repeated induced writes on a chosen topic reveal which partitions receive records on that topic. Because partitions are assigned by content rather than by time, the resulting partition-to-topic map describes the earlier records too. A transfer lemma makes this exact, and shows the two cohorts are exposed equally, provided the adversary learns *which partition holds a given earlier record*. A layout whose regions grow with occupancy discloses that in the snapshot, and so does pgvector's `ivfflat` index under an encrypted file system; one of fixed-size sealed regions does not. Much of the paper is an account of how an adversary obtains that membership

• *S. Fegade is an independent researcher.*
E-mail: shreyasf@icloud.com

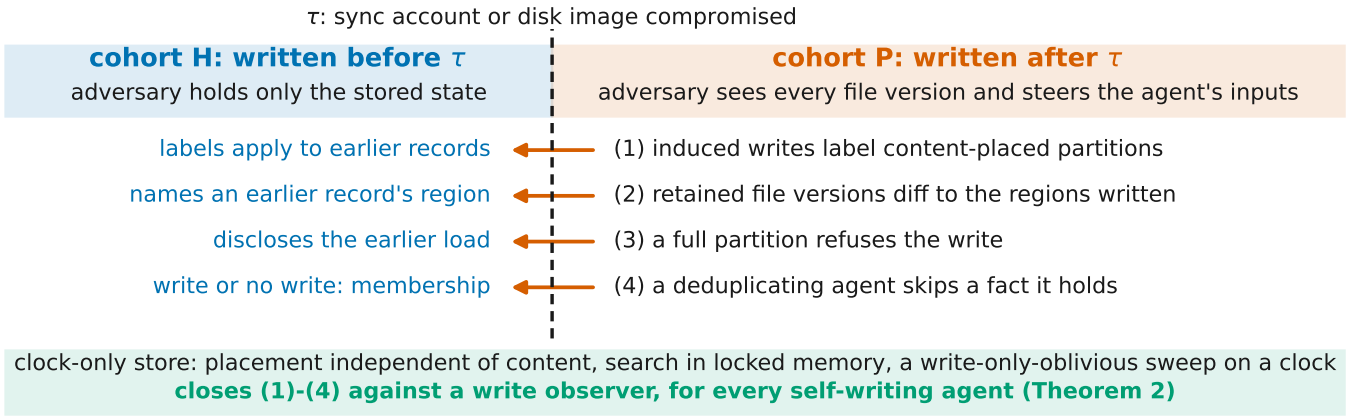


Fig. 1. Historical confidentiality at a glance. The adversary arrives at τ and observes only what happens after it, yet each of four channels turns a post-compromise observation into a statement about records written before the compromise (Section 4). A store whose disk behaviour is a function of the clock alone closes all four against a write observer, and Theorem 2 shows that nothing weaker can close the fourth.

anyway.

What actually protects, and what does not: Subject drift erodes the transfer but does not stop it: on a timestamped issue tracker, in balanced accuracy (which the corpus's skewed topic mix cannot inflate), the advantage falls from 0.217 [0.151, 0.283] to 0.055 [0.027, 0.082] when the induced content is five eras newer. History independence is a single-snapshot notion [9], and two retained versions of a content-placed store diff to exactly the regions written between them; under a real backup tool the diff names an earlier record's region at 13 times the chance rate. A store that bounds per-partition capacity and refuses an overflowing write discloses, at AUC 1.00, that a topic's load has reached capacity, and therefore what the earlier records contributed to it. And an agent leaks through its own operations, down to the occurrence of a write: a memory pipeline that skips a fact it already holds turns deduplication into a membership oracle for facts written before the compromise, which we measure at AUC 0.83 in the unmodified mem0 library; it is open exactly when the deciding model restates a stored fact verbatim, as our characterization predicts.

The design principle: *Semantic indexing need not imply semantic physical placement.* The adversary cannot read the unlocked process's memory and no remote server need hold the index, so encrypted on-disk search is a choice, not a requirement. Dropping it yields a store whose disk layout carries no content: placement independent of content, search in locked memory over a logical partition index, and writes by a clocked sweep in the manner of write-only ORAM. It closes all four channels against a write observer, at 68.4 KiB per flush; placed under the unmodified mem0 library it takes the membership oracle from AUC 0.83 to 0.47. We prove more: against a self-writing agent, a store is secure exactly when what it reveals is independent of what the agent does, so the clock-only schedule is not one sufficient design among many but the shape every secure design must have. Keeping headers on disk extends the store past the memory limit with its writes still invisible to a write observer; only a read observer needs oblivious *search*, which we price rather than claim.

The contributions are as follows.

1) **Historical confidentiality and its threat model:** a τ-arrival adversary split into write-observing (W) and read-write (RW) classes, parameterized by version-capture granularity and audited against the artifacts a real operating system retains (Sections 3 and 7.11).
2) **Structure transfer and the channels that supply its precondition:** the transfer attack, evaluated on deployed stores under file-level encryption and on agent-written records, and three channels that reach the earlier cohort when the layout does not: multi-version reconstruction, refusals, and derived writes, the last a deduplication membership oracle measured in unmodified libraries under four models (Sections 4 and 7).
3) **Dual-epoch security:** an adaptive, leakage-parameterized definition; sufficient conditions that cover overflow; separations from forward/backward privacy, breach resistance and history independence; and a reactive variant with an exact characterization of the stores secure against a self-writing agent (Sections 5, 7.4 and 7.5).
4) **A design space, implemented and audited:** three designs and a larger-than-memory variant, a byte-level auditor, and registered measurements of every channel, including those that stay open (Sections 6 and 7).

Sections 2 and 3 cover related work and the threat model, Section 4 the attacks, Section 5 the theory, Section 6 the designs, Section 7 the measurements, and Section 8 the limitations.

2 Background and Related Work

2.1 Searchable Encryption and Leakage

Searchable symmetric encryption allows a client to query an encrypted index. Its security is stated relative to a leakage function that describes what the scheme is permitted to reveal, typically the access pattern (which encrypted records a query touches) and the search pattern (whether two queries are equal) [10], [11]. A large body of work shows that such leakage can be exploited. Islam, Kuzu, and Kantarcioglu

recovered queries from access-pattern co-occurrence [12]; Cash et al. introduced the count attack [13]; and Blackstone, Kamara, and Moataz reduced the auxiliary knowledge these attacks require [14]. Volume leakage alone suffices for reconstruction [15], [16], and query recovery survives obfuscation [17]. Kornaropoulos, Papamanthou, and Tamassia recover data from the leakage profile of k-nearest-neighbor queries [18], the query type an encrypted vector store serves. Volume-hiding constructions pad response sizes to remove this channel [19], [20]. Xu et al. showed that leakage-abuse attacks remain effective against schemes that are both forward and backward private [21]. Local SSE bounds how scattered a query's reads are on the client's own disk [22], not what a compromise of that disk reveals about earlier writes.

2.2 Forward Privacy, Backward Privacy, and Injection

Forward privacy requires that an update cannot be linked to previously issued search tokens [23], [24]. It was motivated by file-injection attacks, in which an adversary who can insert chosen documents recovers queries from the documents they return [25]. Sophos achieves forward privacy with a trapdoor permutation and a per-keyword counter [24]; Diana replaces the public-key operation with a constrained pseudorandom function built from a GGM tree [26]. Backward privacy limits what searches reveal about deleted entries and is classified into Types I to III [26]; Ghareh Chamani et al. give constructions across this range [27]. Amjad, Kamara, and Moataz generalize forward privacy to injection security [28]. These notions are defined over the transcript of operations observed by a persistent adversary. In prior injection attacks the adversary supplies documents verbatim; here it controls only the task that causes the agent to write.

2.3 Snapshot Adversaries and History Independence

A snapshot adversary observes the stored state at one or more points rather than the full transcript; Fuller et al. systematize it against the persistent adversary of searchable encryption [29]. Amjad, Kamara, and Moataz define breach-resistant structured encryption, which requires that a snapshot adversary learn less than a persistent one [30]. Du, Genkin, and Grubbs construct snapshot-oblivious RAMs [31], and Persiano and Yeo establish lower bounds for such constructions [32]. Grubbs, Ristenpart, and Shmatikov observe that real database snapshots also contain logs, caches, and other artifacts that leak past operations [33], and, attacking applications built on encrypted data, show that a snapshot combined with the application's own behaviour recovers far more than either alone [34] — the composition this paper formalizes for the storage layer. Czeskis et al. showed the same of encrypted and deniable file systems: the operating system and applications around a volume record what it holds [35], and deniable storage has since been designed against an adversary holding several snapshots [36]. Below the file system an SSD's flash translation layer retains superseded pages after an in-place rewrite [37], which is why secure deletion is hard [38]. A synchronization provider is the adversary that sees the most versions, and end-to-end encrypted cloud storage is far from robust against a malicious one [39]; we assume its encryption holds and ask what remains. History independence, anticipated by Micciancio [40] and defined by Naor and Teague [9], requires that a data structure's representation reveal nothing about the sequence of operations beyond its current contents; Buchbinder and Petrank establish its inherent costs [41]. It has been applied to file systems [42] and combined with oblivious storage and secure deletion [43], and Trabucco et al. give a practical history-independent index for queryable-encrypted databases that scales to millions of records [44]. An HI content-placed index still sends a topic's records to that topic's region, so it can be history-independent and dual-epoch *insecure* (Proposition 3). Write-only ORAM hides *writes* rather than reads: HIVE [45], DetWoORAM [46], and ObliviSync [47], which uses fixed-rate syncs to hide write occurrence and timing for synchronized folders. That is the anchoring setting of this paper, and Mnemos-M's sweep is a write-only-oblivious store specialized to agent memory; what is new is its combination with an in-memory semantic index, the derived-write analysis, and the reactive characterization (Theorem 2). Oblivious RAM [48], [49] hides all access patterns at polylogarithmic cost. Dual-epoch security differs from breach resistance in comparing two cohorts of data under one adversary rather than two classes of adversary (Section 5.4).

2.4 Private Vector Search

Compass performs encrypted semantic search by running ORAM over an HNSW graph with client-side traversal [50]. Opal protects personal AI memory by confining data-dependent reasoning to a trusted execution environment while an untrusted disk sees only fixed, oblivious accesses; it handles continuous ingestion by piggybacking reindexing and capacity management on each ORAM access [51], so an oblivious store is not limited to static corpora. What closes the derived-write channel there is oblivious access under a fixed budget, not the enclave as such (Proposition 6). ONYX reduces the cost of disk-oblivious nearest-neighbor search [52]. Pacmann [53] and Panther [54] address private approximate nearest-neighbor search in client-server settings. These systems protect against a compromised server or storage provider and, in Opal's case, assume that the enclave is not compromised, an assumption that transient-execution attacks have violated in practice [55]. None of them distinguishes records by the time at which they were written relative to a compromise.

2.5 Security of Agent Memory

Most self-writing memories store records in plaintext, and the attacks of Section 1 either target integrity or, like MEXTRA [6] and membership inference against chat-agent memory [7], query the running agent, which access control addresses; membership inference against retrieval-augmented generation is likewise a query-side attack [56]. A recent survey confirms that the storage-layer confidentiality of a self-writing memory has not been treated [57]; this paper considers that adversary.

3 System and Threat Model

3.1 Self-Writing Agent Memory

The system under study is an agent that stores memory records locally. Each record consists of text, optional meta-

data, and an embedding vector produced by a public embedding model. Retrieval returns the records whose embeddings are most similar to a query embedding. Four properties distinguish this workload from the usual setting of searchable encryption, in which a user outsources a document collection to a remote server.

Records are *self-authored*, produced as a by-product of the agent's work and never reviewed, and *continuous*, written throughout the life of the store rather than in an initial bulk load. They are *inducible*: a party who can supply tasks to the agent, through a web page, a ticket or an email, causes records to be written without any access to the store, and those records are genuine agent output that content filtering cannot separate from organic ones. And the store is *local*, so the client/server distinction on which the security analysis of schemes such as Sophos [24] relies does not apply: both reside on the same disk.

With no remote server that must hold the index, the disk can be removed from the query path altogether; Section 6.3 takes that option.

3.2 Partitioned Encrypted Vector Indexes

An inverted-file (IVF) index divides the embedding space into B partitions using a table of centroids, called the *partition table*. A record is assigned to the partition with the nearest centroid. A search computes the k_{probe} partitions nearest to the query, scores the records they contain, and returns the top_k best matches. Because partitions are defined by embedding similarity, records on similar topics share partitions.

An encrypted variant seals each record, so that record contents require a key. Writes are typically buffered and flushed in batches of w records, called *epochs*. The analysis distinguishes two storage layouts. In an *unpadded* layout, each partition occupies a file region whose size is proportional to the number of records it holds. In a *padded* layout, each partition occupies exactly C slots regardless of occupancy. A forward-private scheme additionally maintains per-partition state such as update counters. Cohort H is the records written before the compromise time τ and cohort P those written after; $\mathrm{Adv}_{\mathrm{hist}}$ and $\mathrm{Adv}_{\mathrm{post}}$ are topic-recovery advantage on each. The supplementary material tabulates the notation.

3.3 The τ-Arrival Adversary

The adversary gains access to the stored state at an arbitrary time τ and retains it. Before τ it observes nothing directly. At τ it obtains a copy of the on-disk state. After τ it observes storage-level activity and may induce writes up to a budget β. It cannot read the memory of the agent process while the store is unlocked and cannot extract keys from the hardware key store.

What it observes after τ depends on how it obtained access, so the paper distinguishes two classes.

- **W, the write observer.** A compromised synchronization or backup account, or repeated disk imaging. It sees a sequence of file *versions*, so it learns which byte ranges changed between captures and when. It does not see reads at all: no search a local process performs produces a new file version. This is the class the anchoring instantiations belong to.

TABLE 1
What the two adversary classes observe. W is the class the sync, backup and imaging instantiations belong to; RW additionally sees reads. g is the capture granularity.

Observable	Cohort H ($t < \tau$)	Cohort P ($t \geq \tau$)
On-disk state	at τ, plus retained versions	every version, at g
Write offsets	via retained versions	complete, at g
Read offsets	none	RW only
Write timing	via retained artifacts	to the flush schedule
Induced writes	none	up to budget β
Keys, process memory	none	none

- **RW, the read-write observer.** A storage component with file-system visibility but no code execution as the user, principally a network-share or remote-filesystem server that mediates the store's reads and writes. It sees the same writes and additionally every read offset. Search-side leakage matters only for this class. We deliberately do *not* instantiate RW as a malicious local storage driver: a kernel-mode driver can read the unlocked process's memory, which violates A3 and collapses the distinction the model is built on, so it is out of scope rather than a weaker RW.

Table 1 summarizes the two views; Section 7 labels every result with its class. The classes and the version-retention, OS-artifact and hibernation capabilities below are distinct powers: a sync account retains versions but sees no reads, a network-share server sees reads but retains no history, and an imaging adversary obtains artifacts but observes nothing live. A design is credited with closing a channel only against the class that can observe it.

Why induction rather than exfiltration: An adversary who can put content in front of the agent could instead try indirect prompt injection [58]. The two abilities come apart where this attack applies. A memory pipeline such as mem0's extraction step has *no tools and no egress*: it reads the document, decides what to store, and writes, so injected instructions have nothing to act on. And injection defenses — egress gating, hardened summarizers, sandboxes that block tool use on untrusted content [59] — stop the agent from *acting* on content, not from *remembering* it. Where injection is undefended it dominates; the storage channel is the residual threat where it is defended.

Hardware key custody (Section 6.4) protects a copy read offline; malware running as the user can ask the key store to unwrap the master key, and is out of scope for every storage design.

Capture granularity: A synchronization client coalesces changes at an interval set by the client and network, not the store, so the attacks are parameterized by the capture granularity g, the number of flushes between versions the adversary obtains; coarser capture weakens every write-channel attack against W.

3.4 Pre-τ State a Real System Retains

Ordinary artifacts record activity before τ: the NTFS change journal records when the store's files were written, and shadow copies, file-history snapshots and an SSD's flash

translation layer, which retains superseded pages [37], preserve earlier contents (Section 7.11 audits them). The model therefore grants the adversary the state at τ *and* whatever earlier state these artifacts preserve. History independence in the sense of Naor and Teague [9] is a single-snapshot notion and does not bound what a *sequence* of representations reveals; a synchronization account that retains versions moves the effective compromise boundary back to the start of its retention window (Section 4.6, countermeasure in Section 6.2).

Hibernation: A hibernation image captures a locked arena and the unwrapped master key, since locking pages prevents paging but not hibernation. Locking the store and zeroizing its arena on suspend and hibernate notifications is therefore a deployment requirement, and the guarantee is conditional on the operating system delivering that notification.

3.5 Assets, Goals, and Assumptions

The assets, in decreasing order of sensitivity, are the content of stored records, their topics, the temporal pattern of the agent's activity, and the existence of specific records; a record's topic, meaning the subject matter the agent worked on, is often as sensitive as its text and is easier to recover. The security goals are content confidentiality, topic confidentiality for cohort P, query privacy under observation of storage access, resistance to correlation of induced writes, and topic and membership confidentiality for cohort H, which is the focus of this paper.

The analysis assumes secure cryptographic primitives (A1); a host uncompromised before τ (A2), without which cohort H does not exist and the problem reduces to that of a fully observed store; that the adversary neither defeats hardware key custody nor reads the memory of the unlocked agent process (A3), which is what makes a memory-resident design possible at all and which Section 6.3 takes at face value rather than paying for on-disk search the threat model does not require; a public embedding model, with no security derived from its secrecy (A4); and a finite injection budget (A5), as a function of which results are reported. Availability, the integrity of agent behavior, microarchitectural side channels, and deployments spanning multiple hosts are out of scope.

Passive versus active storage: The adversary of this paper is passive at the storage layer: it reads versions and induces writes through the agent, but does not itself modify, roll back, or fork the stored files. A compromised synchronization account could do all three, and tampering is a membership channel in its own right — corrupt the region an earlier record would occupy and watch whether the agent later errs on that topic. We scope tampering out of the confidentiality analysis for a stated reason: every slot is an authenticated ciphertext (S2), so a tamper surfaces as an authentication failure. Mnemos-M reads the whole store at unlock and fails closed, independently of content; Mnemos-P fails when a search probes the region, which the RW class observes. Authentication does not detect rollback to an authentic earlier version; a hardware monotonic counter bound into the manifest's associated data is the standard defense, and we leave it to future work.

4 Attacks on Self-Writing Memory

This section describes the attacks. The first exploits a partition layout whose regions grow with occupancy, and it motivates the security definition; the others exploit channels that survive a layout built to close it. Experimental results for all of them are reported in Section 7.

4.1 Leakage of a Growing Partition Layout

An IVF index is naturally stored with the records of each partition in a contiguous region, because a search reads whole partitions and contiguous storage makes those reads efficient. When regions grow as records are added, the snapshot at τ reveals the size of every partition and the region that holds each sealed record. After τ, the offsets of writes reveal which partitions receive new records. Neither observation requires a key: encryption hides the content of a record but not its location.

4.2 Structure Transfer

The attack consists of three steps.

Induction: The adversary selects a topic t from a known set and causes the agent to receive a task on t, such as a document to summarize or a report to triage. The agent writes a record in its own words. The adversary knows the topic and the approximate time of the induced record, but neither its content nor its partition; unlike classical file injection [25], it does not choose the stored text. That supplied content causes a persistent memory write is documented against deployed assistants [60], [61] and across memory backends [62], and Section 7.3 measures it for a mem0-style pipeline [63] driven by local models.

Localization: A flush of w records reveals the set of partitions that received a record. The adversary repeats the induction and combines the observed sets with one of two estimators.

Intersection. If several induced records are known to share a partition, intersecting the partition sets of their flushes isolates it; with the other $w-1$ records uniform, an unrelated partition survives r intersections with probability q^r, $q = 1 - (1 - 1/B)^{w-1}$, so batching raises the cost without preventing the attack. The adversary cannot generally tell which induced records share a partition, so this estimator is retained only as an upper bound.

Frequency analysis. Without attribution, the adversary computes, for each topic t and partition b, the fraction of flushes carrying an induced record on t in which b receives a write. A partition holding records on t receives writes in the flushes of t more often than in the flushes of other topics. Each partition is labeled with the topic for which this rate is highest, provided the rate exceeds the mean over all topics by twice the estimated sampling noise. Padding writes to uniformly random partitions raise every rate by the same expected amount; they add noise the adversary must overcome with a larger budget, but they do not remove the difference between topics.

Transfer, and what it requires: The adversary assigns each cohort-H record the label of the partition that holds it. By Lemma 1 the expected accuracy of this assignment is the same on cohort H as on cohort P, so the attack exposes

the two cohorts equally, except to the extent that the agent's subject matter has drifted between them or the adversary's taxonomy differs from the user's.

The step that makes this work is the lemma's third hypothesis, knowing which partition holds a given cohort-H record, which a growing layout supplies in its snapshot and a sealed fixed-size layout does not. The rest of this section is an inventory of channels that supply it anyway.

4.3 Deployed Stores, and Time-Placed Controls

Deployed vector stores place records by content (FAISS and pgvector's `ivfflat` by inverted list, Chroma by HNSW neighbourhood, LanceDB by IVF-PQ list) and persist that placement in files whose changed byte ranges a size-preserving encryption layer such as `gocryptfs` or Cryptomator does not hide. Placing by content is necessary for the attack but not sufficient: the layout must also disclose *which* partition holds a stored record at the observer's granularity, which a store that rewrites its whole index on every save does not, and a graph index only through structure and insertion order. Stores that place by arrival time, such as an unindexed LanceDB table, serve as negative controls; Section 7.2 measures all of them.

4.4 The Refusal Channel

A store that bounds per-partition capacity must do something when a partition fills. If it refuses the write and writes nothing, the refusal is observable, and by Proposition 5 what it discloses is the partition's live load, which is dominated by cohort H once the adversary accounts for its own induced contribution. The attack is direct: induce writes on a topic and watch whether the region that topic's records occupy is rewritten. Section 7.7 measures it, and measures the padded refusal that closes it.

4.5 Derived Writes and Deduplication Membership Inference

Proposition 6 says a store whose operations depend on the content *or the occurrence* of what the agent does leaks cohort H to a feedback agent. The occurrence half admits a concrete attack. Deduplication as a membership side channel is not itself new: Harnik, Pinkas, and Shulman-Peleg showed that cross-user deduplication in cloud storage lets one user learn whether another has already stored a file [64], a leak message-locked encryption accepts by design [65]; shared prompt caches in language-model serving leak cached prompts in the same way [66]. What is new here is the instantiation in self-writing agent memory, where the adversary supplies the "file" as a document for the agent to remember, the deduplication decision is made by the agent's own extraction pipeline, and the disclosure is dated relative to a compromise.

Agent memory pipelines avoid storing a fact twice, and they do so by consulting what is already stored. An adversary that supplies a document and can see whether a write happened therefore learns whether a matching fact was already in memory. Because the adversary chooses the document it chooses the fact, and a fact present before τ is a cohort-H membership disclosure. Nothing is decrypted; the signal is the occurrence of a write. Against a store that flushes when records are staged, the occurrence of a flush is the answer. Against one that flushes on a clock, a flush happens either way and the channel is closed. Section 7.3 measures both sides.

The precondition holds beyond our prototype. Reading the shipped source of nine open-source agent-memory frameworks, four make whether a write reaches storage depend on whether the content is already stored: mem0 drops an extracted fact whose hash is already present, CrewAI writes a record only if a model's consolidation plan says *insert*, LangMem's manager creates only novel memories, and Graphiti, the engine of Zep [67], deduplicates entities against the graph. The supplementary material gives each framework's version and the file and line of the mechanism. A mechanism in the source is necessary, not sufficient: the write must also depend on membership when the pipeline runs, which Section 7.5 measures for mem0 and LangMem under several models.

4.6 Multiple Versions

An adversary that retains several versions of the store file diffs consecutive versions and reads off the regions rewritten in between. This is the write channel applied *before* τ, and it is what supplies the cohort-H membership that Section 4 otherwise lacks: regions labeled after τ then give those earlier writes their topics. The attack is parameterized by the capture granularity g of Section 3.3, and Section 7.7 reports it as a function of g, together with the session-boundary re-encryption that defeats it when the adversary cannot capture more finely than a session.

5 Dual-Epoch Security

The attack of Section 4 exposes cohort H almost as much as cohort P. This section explains why, defines the notion that separates the two cohorts, gives sufficient conditions for it, places it against existing notions, and extends it to an agent whose writes depend on what it retrieves.

5.1 A Transfer Lemma

The finding does not depend on a particular attack; it follows from the placement rule. The lemma is at bottom an exchangeability observation, and its content is in the *third* hypothesis. It states structure transfer as a property of *composed* leakage: two leakages that are each independent of cohort-H topics, the partition of each stored record and the labels induced writes attach to partitions, together determine those topics.

Lemma 1 (Transfer)**.** *Let records be assigned to partitions by a function of content alone, let cohorts H and P be drawn from the same distribution over content, and let the adversary observe the partition of each stored record. For any partition-labeling ℓ that the adversary computes without reference to cohort membership, the expected topic-recovery accuracy of ℓ on cohort H equals its expected accuracy on cohort P.*

Proof sketch. A record's partition, and hence its predicted label, depends only on its content, as does its true topic. Whether ℓ is correct on a record is therefore a function of content, whose distribution is the same in both cohorts. □

Applying a partition label to a cohort-H record requires knowing which partition holds it. A growing layout discloses that in the snapshot; against fixed-size sealed regions the lemma says nothing. The write channel of Section 7.7 is therefore a statement about cohort P unless another channel supplies cohort-H membership, and Section 7.7 tests one. Subject drift breaks the second hypothesis, which Section 7.1 measures.

5.2 The Security Game

Dual-epoch security is defined for a leakage function $\mathcal{L} = (\mathcal{L}^{\mathrm{snap}}, \mathcal{L}^{\mathrm{op}})$: the state exposed at τ, and what each post-τ operation reveals. The leakage is stated over *plaintext operations*, as is usual for structured encryption; stating it as "the regions the operation touches" would let the transcript reveal itself and make the theorem empty. Let π be a secret relabeling of partitions, sampled once. Then $\mathsf{put}(x)$ reveals $\pi(\mathrm{part}(x))$; $\mathsf{search}(q)$ reveals π of the probe set of q and the fresh random extras of that call; delete and update reveal nothing when called, and an update reveals the new record's relabeled partition at its next flush; every operation reveals its time. The simulator samples π itself, so the keyed permutation buys consistency of the relabeling and nothing more (Section 7.8 measures what that is worth against a read observer).

The game $\mathbf{G}^{\mathrm{H}}_{\Pi,\mathcal{A}}$ for a dynamic encrypted memory Π runs as follows.

1) $\mathcal{A}$ outputs two histories σ_0, σ_1 of equal length, each a sequence of put and delete operations. Either may drive a partition past capacity.
2) The challenger samples $b \leftarrow \{0,1\}$, initializes Π, executes σ_b, and flushes, so no record of σ_b remains staged. This point is τ. $\mathcal{A}$ receives $\mathcal{L}^{\mathrm{snap}}$.
3) $\mathcal{A}$ issues put, delete, update and search operations *adaptively*, each chosen after the leakage of the earlier ones, and may reference records of σ_b. After each it receives $\mathcal{L}^{\mathrm{op}}$, never a search result.
4) $\mathcal{A}$ outputs b'; its advantage is $\mathsf{Adv}^{\mathrm{H}}_{\Pi,\mathcal{A}}(\lambda) = |\Pr[b' = b] - \frac{1}{2}|$.

Definition 1 ($\mathcal{L}$-dual-epoch security)**.** Π is $\mathcal{L}$-dual-epoch secure if for every probabilistic polynomial-time $\mathcal{A}$ there is a simulator $\mathcal{S}$ such that the view of $\mathcal{A}$ in $\mathbf{G}^{\mathrm{H}}_{\Pi,\mathcal{A}}$ is computationally indistinguishable from the output of $\mathcal{S}$ given only $\mathcal{L}$ of the operations $\mathcal{A}$ issued; consequently $\mathsf{Adv}^{\mathrm{H}}_{\Pi,\mathcal{A}}(\lambda) \le \mathsf{negl}(\lambda)$.

The simulator sees nothing about σ_b, so the entire view is independent of which history produced cohort H. The flush at τ matters because a staged record would otherwise be written in the adversary's view; equal lengths matter because the maintenance cursor and global occupancy count pre-τ operations. Adaptivity matters because the τ-arrival adversary induces writes in response to what it has seen; the hybrid proof is unaffected, since each step replaces a primitive rather than fixing the operation sequence.

Corollary 1 (Predicate hiding)**.** *If Π is $\mathcal{L}$-dual-epoch secure, then for any predicate ϕ on cohort-H contents, in particular the topic of any record, no adversary predicts ϕ from its view with advantage over the best cohort-oblivious predictor exceeding $2\,\mathsf{Adv}^{\mathrm{H}}_{\Pi,\mathcal{A}}(\lambda)$, averaged over the distribution from which the histories are drawn.*

The argument is the usual equivalence of semantic security and indistinguishability (supplementary material). A measured advantage is a *lower* bound on what one classifier extracts, so Section 7.4 reports the upper end of each interval as the ceiling for the attacker it ran and claims nothing beyond it.

5.3 Sufficient Conditions

Theorem 1. *Let Π store records in B partitions of fixed capacity C, in a single file of B equal regions, and let it satisfy the following conditions.*

- ***S1 (fixed size).*** *Every file the store maintains has a length determined by the public parameters alone.*
- ***S2 (sealed slots).*** *Every slot is encrypted with an authenticated cipher that is IND$-CPA and INT-CTXT, under a fresh 192-bit random nonce; a free slot is uniformly random bytes; and any cleartext index within a slot is the output of a pseudorandom function of a uniformly random identifier.*
- ***S3 (data-independent updates, with padded refusal).*** *Each flush rewrites exactly $w + 1$ whole regions: one per staged record, uniformly random regions for the remainder up to w, and one region chosen by a counter that depends only on the flush index. A deletion writes nothing until the next flush.* A write whose partition is full is refused to the caller but performs the same region rewrite an accepted one would have. *The manifest is rewritten every flush regardless of content.*
- ***S4 (independent placement).*** *The partition table is independent of cohort H.*

Then Π is $\mathcal{L}$-dual-epoch secure for the $\mathcal{L}$ above, with $\mathcal{L}^{\mathrm{snap}}$ empty beyond the public parameters, for every pair of equal-length histories, including pairs that overflow a partition.

Proof sketch. $\mathbf{G}_1$ replaces every pseudorandom function with a random one; $\mathbf{G}_2$ replaces every ciphertext, live or free, with a uniform string of the same length (IND$-CPA; S1 makes the lengths match). In $\mathbf{G}_2$ the snapshot is fixed-length uniform strings, independent of b. Each post-τ flush writes the regions S3 names: the staged records' relabeled partitions, uniform choices and a counter, none of which depends on the history behind the records already present, and by S4 neither does the table. A write refused under one history and accepted under the other performs the same rewrite in both, so overflow does not distinguish. The view is a function of $\mathcal{L}$ and public randomness. The full proof, with the concrete bound, is in the supplementary material. □

Padded refusal is what lets the theorem cover overflow: a store that rewrites the intended region whether or not the write was admitted makes the two observationally identical. Overflow then costs availability, not confidentiality, since the agent's write is discarded; Section 7.7 measures how many writes a skewed trace loses, and that the first refusal arrives at 23.8% of nominal capacity.

Whole-region rewrites make a write reveal which region changed and never which slot. A per-slot write need not reveal occupancy either, if the slot is chosen well.

Lemma 2 (Slot-index independence)**.** *Let each accepted record occupy a slot drawn uniformly from the free slots of its region, and*

TABLE 2
What each notion evaluates. Snapshot security and history independence judge a single stored state; forward/backward privacy judges the operation transcript; breach resistance judges one snapshot with a transcript. Dual-epoch security judges a snapshot and the post-compromise transcript *jointly*, across two cohorts; its reactive form adds the agent's control flow. Columns: snapshot security (Snap.), history independence (HI), forward/backward privacy (FP/BP), breach resistance (BR), dual-epoch security (DE).

	Snap.	HI	FP/BP	BR	DE
One snapshot	✓	✓	—	✓	✓
Multiple snapshots	—	partial	—	partial	✓
Post-τ transcript	—	—	✓	✓	✓
Chosen inputs (adaptive)	—	—	✓	partial	✓
Two cohorts, one adv.	—	—	—	indirect	✓
Reactive-agent flow	—	—	—	—	RDE

let nothing else in the store depend on slot labels within a region. For any pair σ_0, σ_1 that never overfills a partition and any region r, the sequence of slot indices at which post-τ records are placed in r has the same distribution under $b = 0$ and $b = 1$.

The proof is exchangeability of slot labels (supplementary material). The lemma is as strong as its hypothesis, which fails for maintenance: freeing tombstoned cohort-H records slot by slot writes as many slots as were deleted, so the purge stays a whole-region rewrite. Nor does it hide *whether* a write happened, which a clocked flush must: a slot written twice with no deletion between was padding the first time. Section 7.4 measures all four combinations of placement rule and write granularity.

5.4 Relation to Prior Notions

Table 2 places dual-epoch security against existing notions, and the results below are the separations; constructions and reductions are in the supplementary material. In one sentence, dual-epoch security is *storage forward secrecy for structure*: it bounds what a compromise at τ reveals about the placement of data written before τ.

Proposition 1 (Forward and backward privacy are insufficient)**.** *There is a scheme that is forward and Type-II backward private but not dual-epoch secure.*

The witness adjoins a per-partition put counter to the client state of any such scheme. Transcript notions are blind to it, but the snapshot's multiset of counters separates m puts to one partition from $m/2$ to each of two. Such state (Sophos's counters [24], Diana's GGM keys [26]) leaks little on a realistic corpus (supplementary material); the existence separation and its magnitude are separate claims.

Proposition 2 (Breach resistance is implied, and insufficient)**.** *Dual-epoch security with snapshot leakage $\mathcal{L}^{\mathrm{snap}}$ implies breach resistance with respect to $\mathcal{L}^{\mathrm{snap}}$. The converse fails, even together with forward and backward privacy.*

Restricting a dual-epoch simulator to the snapshot gives a breach-resistance simulator; the witness of Proposition 1 separates the converse. Neither notion constrains the *joint* object, a snapshot read with the writes an adaptive adversary induces after it, and that object is where structure transfer lives.

Proposition 3 (History independence is insufficient)**.** *A store that is strongly history-independent in the sense of Naor–Teague need not be dual-epoch secure, even against a single post-τ snapshot.*

History independence hides the order in which a set was inserted, not which set was inserted; a content-placed HI store still keeps a topic in that topic's region.

Proposition 4 (Sealing and padding are each necessary)**.** *Slot-level history independence with regions that grow with occupancy is not dual-epoch secure, and padding regions to a fixed size without sealing free slots is not dual-epoch secure.*

The first leaks occupancy through region sizes, the second through the boundary between written and unwritten slots; Section 7.4 realizes both as ablations.

Version retention as a leakage parameter: The multi-version channel lies outside Definition 1, whose snapshot is one state. Let $\mathcal{L}_g^{\mathrm{snap}}$ additionally reveal the file's versions every g flushes before τ; *g-retained dual-epoch security* is Definition 1 with $\mathcal{L}_g^{\mathrm{snap}}$. Security at $g = \infty$ does not imply it at finite g: Mnemos-P meets Theorem 1, yet its version diffs name a pre-τ record's region (Section 7.7). A clock-only store (Theorem 2) is g-retained secure for every g, since a diff at any granularity reveals only which cursor ranges were swept. The same asymmetry decides what an imaging adversary gains from superseded flash pages.

5.5 Capacity and Confidentiality

Proposition 5 (Capacity–confidentiality trade-off)**.** *Let Π place records by content into partitions of capacity C, and let a write whose partition is full be refused without performing the writes an accepted record would have performed. Then an adversary that can induce writes on a topic and observe the operation leakage learns whether that topic's live load has reached C, and hence the load contributed by cohort H. Dual-epoch security therefore requires either that C exceed the worst-case per-topic load, or that a refusal be indistinguishable from an acceptance.*

An accepted put reveals $\pi(\mathrm{part}(x))$ and a silently refused one reveals nothing, so the distinguishing event is exactly "this topic is at capacity." Padded refusal (S3) takes the second horn and pays in lost writes; over-provisioning takes the first and pays in provisioning for the most frequent topic, which under skew is most of the file. Section 7.7 measures both. A design with only a *global* bound, such as Section 6.3, escapes the proposition for a different reason: its disk schedule is a function of the clock alone (Theorem 2), so a refused write changes nothing that is written.

5.6 Reactive Security and the Derived-Write Channel

In Definition 1 the adversary chooses every post-τ operation. A self-writing agent is a second source of operations: it retrieves a cohort-H record and, in the same turn, writes or searches about the same subject, so a post-τ operation is a function of cohort-H plaintext that the storage adversary cannot read. An adaptive adversary does not have that secret-dependent control flow, so this is a separate model: secure storage composed with a reactive program.

The reactive game: A *feedback agent* P is a stateful polynomial-time program that, on each turn, receives an input, may search Π and read the plaintext results, and then issues put, delete or update operations. The game $\mathbf{G}^{\mathrm{R}}_{\Pi,\mathsf{P},\mathcal{A}}$ runs as $\mathbf{G}^{\mathrm{H}}$ through τ; then the challenger runs P on inputs the adversary supplies one at a time, at times and between unlocks and locks the adversary chooses, and after each turn the adversary receives $\mathcal{L}^{\mathrm{op}}$ of the operations P issued and nothing else: no output of P, no search result. P controls neither the clock nor the session boundaries, so the schedule is not itself a channel from the agent. $\mathsf{Adv}^{\mathrm{R}}_{\Pi,\mathsf{P},\mathcal{A}}$ is defined as before.

Definition 2 (Reactive dual-epoch security)**.** Π is *reactively $\mathcal{L}$-dual-epoch secure* (RDE) if for every polynomial-time feedback agent P and every polynomial-time $\mathcal{A}$ there is a simulator with the view of $\mathbf{G}^{\mathrm{R}}$, so that $\mathsf{Adv}^{\mathrm{R}}_{\Pi,\mathsf{P},\mathcal{A}}(\lambda) \le \mathsf{negl}(\lambda)$.

Proposition 6 (Derived-write separation)**.** *Let Π have post-τ operation leakage $\mathcal{L}^{\mathrm{op}}$ that reveals a non-constant function of the content of an operation, or of whether an operation occurred at all. Then there is a feedback agent* P *against which Π is not RDE, even if Π is dual-epoch secure by Definition 1.*

Proof. Let P be a deduplicating writer: on input x it searches for x and writes x only if the top result is not a near-duplicate. Let σ_0, σ_1 differ in whether a fact $x^\star$ is present, and supply $x^\star$. P writes under one history and not the other, which occurrence leakage reveals; if P always writes, choose histories under which its write lands in different partitions, which content leakage reveals. Mnemos-P flushes on demand and places by content, so it meets Theorem 1 and is not RDE. □

The deduplicating agent is one instance of a general fact: an agent can select between any two behaviours on one bit of what it retrieved, so any dependence of the leakage on what the agent does is a channel, and its absence is enough. Call Π *operation-oblivious* if at every turn the adversary's view of that turn, given its view so far, is computationally indistinguishable whatever operations the turn issues, none included, and whatever history produced the store.

Theorem 2 (Reactive security is operation obliviousness)**.** *Let Π satisfy S1 and S2 and be dual-epoch secure (Definition 1).*

(i) *If Π is operation-oblivious, it is RDE against every feedback agent.*

(ii) *If on some store state two operation sequences O and O', searches included, leak at statistical distance ε, there are a feedback agent* P *and histories with $\mathsf{Adv}^{\mathrm{R}}_{\Pi,\mathsf{P},\mathcal{A}} \ge \varepsilon/2 - \delta - \mathsf{negl}(\lambda)$, where δ is the probability that a search misses a stored exact duplicate.*

(iii) *The following clock conditions make Π operation-oblivious, with leakage $\mathcal{L}_{\mathrm{clock}}$ that reveals only the unlock, lock and flush times.*

- ***C1 (clocked schedule).*** *The set of disk locations written at each flush is a function of the flush index and of a start drawn uniformly at unlock — a deterministic sweep of κ slots per flush — and a flush occurs every Δ while unlocked, and once at lock, whether or not anything was staged.*
- ***C2 (fresh sealed writes).*** *Every written slot is a fresh authenticated ciphertext of the fixed slot length, live or free alike.*
- ***C3 (no post-unlock reads).*** *After the single sequential read at unlock, a* search *issues no disk access; ranking and payload retrieval are served from the locked arena.*

Proof sketch. (i) Replace the operations of each turn, one turn at a time, by the empty sequence; each step is indistinguishable by obliviousness, and the final view, the snapshot and an idle schedule, is independent of b by S1 and S2. (ii) P searches for its input and issues O if a near-duplicate is returned and O' otherwise; σ_0 holds the input and σ_1 does not, and $\mathcal{A}$ applies the optimal test for O against O'. Dual-epoch security makes the search and the two store states indistinguishable on their own, so the test's advantage survives up to the probability δ that P takes the wrong branch. (iii) The two hybrids of Theorem 1 make every written byte uniform; each flush then writes the slot range C1's cursor names, a function of the flush index and not of anything P did, wrote or read, and by C3 no search adds a disk event. The supplementary material gives the full argument, including the sealed journal that carries staged records between flushes on the same clock. □

Part (i) is the definition read in the other direction; the content of the theorem is that nothing weaker suffices (ii) and that the clock achieves it (iii). Two consequences follow. Under RW a search's reads are part of what a turn reveals, and an agent searches about what it has just retrieved, so by (ii) a store whose reads depend on the query is not RDE against RW: only one that reads nothing after unlock, or reads obliviously, is. And the theorem is a property of the store's files: an agent framework that persists anything else whose writes depend on the agent's decisions, such as a history log recording each add or skip, reopens the channel above the store unless that artifact is kept in the store's journal or outside the synchronized and imaged scope.

Corollary 2 (Mnemos-M is reactively secure)**.** *The memory-resident store of Section 6.3 with back-pressure satisfies C1–C3 and is reactively dual-epoch secure against every feedback agent, for both adversary classes. Without back-pressure it meets C1 at every flush except a lock whose backlog exceeds the journal, which reveals that it did. Mnemos-P is not RDE (Proposition 6), so RDE $\Rightarrow$ DE and DE $\not\Rightarrow$ RDE.*

Section 7.5 plays the reactive game against both designs and a graded series of ablations, and finds the clock-only store at chance while every store that flushes on a write or places by content is distinguished. This is what separates self-writing memory from classical searchable encryption, whose queries are external to the stored data: closing the channel needs content-oblivious access *and* a content-independent operation schedule.

6 A Design Space

The designs below sit at different points of the space Theorems 1 and 2 and Propositions 5 and 6 map out. All are implemented in Rust, with a Python binding used only by the evaluation harness and the embedding model.

- **Mnemos-RS:** a preallocated record slab addressed by a pseudorandom function of each record's tag, beside a separate forward-private encrypted IVF index; a search fetches its results from the slab by position. It is the natural design from searchable encryption.
- **Mnemos-P:** one sealed file of B equal regions, placed by content, searched on disk. The construction Theorem 1 is proved for.
- **Mnemos-M:** the same sealed file, placed independently of content, searched in memory, and written by a clocked sweep. **Mnemos-M/disk** keeps headers and payloads on disk for stores larger than memory.

6.1 The Shared Layout

Mnemos-P and Mnemos-M share a file of B equal regions of C slots. A slot holds two authenticated ciphertexts, a fixed-size header carrying the record identifier and its quantized embedding and a fixed-size payload carrying the text and metadata. Every ciphertext uses XChaCha20-Poly1305 under a fresh 192-bit random nonce; a header's associated data binds it to its slot and format version, a payload's to its record. A free slot is uniformly random bytes. The file lengths are functions of B, C, the embedding dimension and the block size, so S1 and S2 hold. The manifest, which holds tombstones and the cursor, is itself one fixed-size authenticated ciphertext.

6.2 Mnemos-P: Placement by Content

A record's region is the keyed permutation of its nearest centroid's partition, so that an adversary holding a public partition table cannot map a topic to a region directly; a secret rotation of the centroids would not do this, since it preserves the nearest-centroid assignment. Against the RW class the permutation is weaker than it appears, because regions read together are geometric neighbours and the co-occurrence graph of reads can be matched to the public table (Section 7.8); against W, which sees no reads, it holds.

A flush rewrites $w + 1$ whole regions (S3), so a write reveals which region changed and never which slot. A record whose partition is full is refused rather than spilled, and the refused write performs the same region rewrite an accepted one would have (Proposition 5). The whole file is re-encrypted at lock, so a version diff spanning a session covers every region, and the maintenance cursor restarts at a random position at each unlock, so its resting position does not count the flushes before τ.

Search: A search reads the regions of its probe set, ranks the decrypted headers in memory, and decrypts the top_k payloads from the regions already read, so no result-dependent access reaches the disk (Section 7.8). Every query reads the same number of regions: by default its k_{probe} nearest partitions plus $(s-1)k_{\mathrm{probe}}$ fresh random ones, or alternatively one of a fixed set of disjoint groups. Neither hides query equality (Section 7.8); only grouping resists permutation recovery.

6.3 Mnemos-M: Placement Without Content, Search in Memory

Assumption A3 grants that the adversary cannot read the unlocked process's memory, and in the local setting no server must hold the index. Mnemos-M takes both at face value.

Search: At unlock the store is read once, sequentially, and every header and payload is decrypted into the locked arena. A search ranks the resident headers and returns resident payloads, so it performs *no disk access*: the search, result and query-equality channels are absent rather than mitigated. Exhaustive ranking is linear in n, so Mnemos-M also keeps each record's nearest-centroid partition in the arena and, when configured to, ranks only the query's k_{probe} nearest partitions. The partitions are *logical*: the disk layout does not change, and neither adversary class observes the index. It costs recall (Section 7.10) and unlock time (Section 7.9).

The sweep flush: Writes follow a deterministic write-only ORAM [46]. Every Δ while the store is unlocked, whether or not anything was written, the flush rewrites the next κ slots after a cursor, each as a fresh sealing: a live slot is re-encrypted, a free slot either receives a staged record or is refreshed with random bytes. A staged record waits in a fixed-size sealed journal, rewritten every flush, until the cursor reaches a free slot. The cursor starts at a random position at each unlock and advances by κ per flush, so the set of slots a flush writes is a function of the flush index and that start (C1); every written unit is a fresh fixed-length ciphertext (C2); and searches touch no disk (C3). Theorem 2 therefore applies. A record's position depends on the cursor and on which slots are free, never on its content, and the occurrence of a flush no longer implies that the agent wrote, which closes the derived-write channel. A staged record reaches its slot within one pass of the cursor, and until then it is durable in the journal.

Throughput and back-pressure: At load ρ a flush passes κ slots of which a fraction $1-\rho$ are free, so the sweep places about $\kappa(1-\rho)$ records per flush and sustains a write rate of $\kappa(1-\rho)/\Delta$; a faster agent builds a backlog. The journal holds J staged records. By default a lock that finds more than J staged flushes until J remain, so the number of lock-time flushes, and only that, depends on the agent: over 40 sessions per setting, no session at a write rate below the placement rate reached it, and 25% to 35% of sessions above it did. With back-pressure a put that would stage more than J records is refused to the caller, which can retry after the next tick; the refusal changes nothing on disk, every lock is one flush, and C1 holds without exception. Back-pressure is to the sweep what padded refusal is to Mnemos-P: availability traded for a schedule that does not depend on the agent.

Capacity: There is no per-partition capacity, so Proposition 5 does not apply. A write refused at the global bound is refused to the caller and changes nothing the clocked sweep writes.

Mnemos-M/disk: For stores larger than memory, headers and payloads stay on disk and the arena holds only the position map and each slot's logical partition, a few bytes per record. Writes are the sweep's, unchanged, so against W every write-side channel remains closed at any size. A search reads the headers of its probed logical partitions, so RW sees

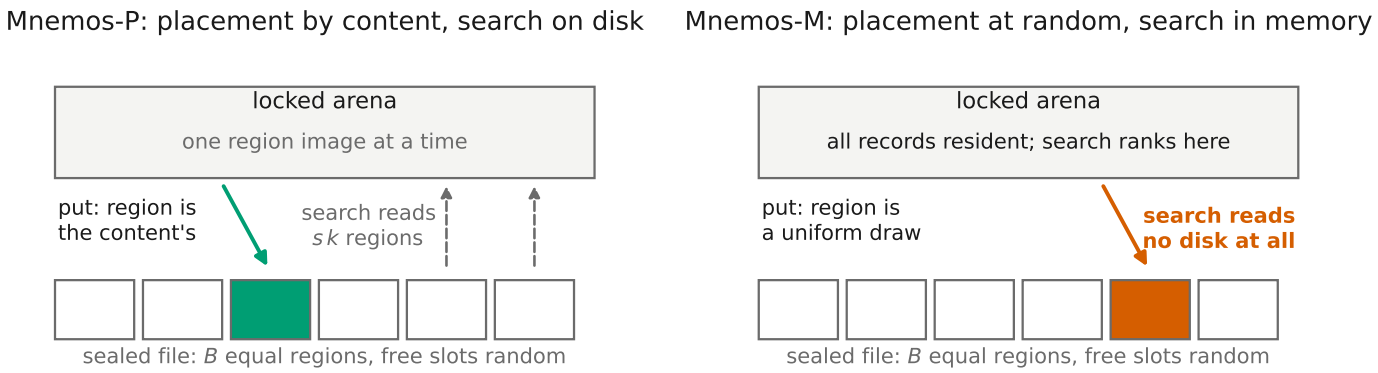


Fig. 2. Where each design puts a record and where it searches. Both use one fixed-size file of sealed slots, so neither reveals a region's occupancy. Mnemos-P sends a record to the region its *content* selects and reads $s\,k_{\text{probe}}$ regions back from disk to answer a query; Mnemos-M places a record independently of its content and answers queries from the locked arena, touching no disk.

which slots were read: the read channel of any on-disk index, which only oblivious search closes.

6.4 Deletion, Key Custody, and Instrumentation

A deletion records a tombstone in the sealed manifest and writes nothing else at the call, so it does not name the record's region; an update is a deletion plus a fresh write. Mnemos-P frees tombstoned slots when its maintenance cursor rewrites their region, within B flushes; Mnemos-M frees them when the sweep reaches them, within one pass of the cursor in wall-clock time. Neither reaches copies a synchronization provider already retains.

The master key is wrapped by a hardware key store, the Windows TPM through the CNG Platform Crypto Provider, and unwrapped into a locked, promptly zeroized region only while the store is unlocked; working keys are derived from it by HKDF. Custody is paired with a guard that locks every store on the operating system's suspend notification (Section 3.4), tested with the broadcast Windows sends before sleep.

Every disk-visible operation emits one typed event naming the file ranges it touched; attacks consume only that stream, and the auditor (Section 7.6) checks it against the disk.

7 Evaluation

The evaluation answers four questions. **RQ1:** does structure transfer reach the earlier cohort in realistic settings, on a timestamped corpus, deployed stores and agent-written memory (Sections 7.1 to 7.3)? **RQ2:** do the conditions of Theorems 1 and 2 hold in the compiled engines, and which are load-bearing (Sections 7.4 to 7.6)? **RQ3:** which channels still reach cohort H once the layout is sealed, and against which adversary class (Sections 7.7 and 7.8)? **RQ4:** what do the designs cost in time, disk volume and retrieval quality (Sections 7.9 and 7.10)? Section 7.11 audits what the operating system retains, and Section 7.12 puts every channel on one scale.

Setup: Records are embedded with `bge-base-en-v1.5` [68]. Corpora are D2, eight BEIR domains [69] used as a topic taxonomy; D3, a synthetic procedural-memory corpus; 13,894 timestamped `kubernetes` issues labeled by their `sig/*` area; 20 Newsgroups; and memories a local model wrote from those issues. Advantage is accuracy minus chance, untruncated, for an adversary that guesses wherever it has no label. Intervals are 95%, over each experiment's independent units: bootstrap from twenty units, Student-t below; the supplementary material names the unit and its count for every experiment and specifies the learned attacker. Attacks read only the engine's emitted event stream, which is the storage adversary's view, and every number in the text is generated from the stored results. An outcome marked *registered* was written into the artifact's verifier, with the check that tests it, before the run, and since the artifact was placed under version control the commit order dates each such prediction. A registered prediction that failed is reported with its numbers where it bears on a claim, and every registered outcome is tabulated in the supplementary material. Advantages are not comparable across experiments, which differ in corpus, chance rate and budget unit; each subsection states its own.

7.1 Structure Transfer, and What Drift Is Worth

The estimators are frequency analysis and an expectation-maximization estimator, neither of which sees the true partition of an induced record; induced records are excluded when scoring, and the intersection oracle is retained only as an upper bound.

On the timestamped `kubernetes` corpus, inducing with content five eras newer than cohort H lowers the advantage on it from +0.42 to −0.02, which is the protection drift is supposed to give. Figure 3 shows what that leaves open: for a public tracker the adversary can induce with archival content, and the advantage depends on the time gap between cohort H and the induced content. Drift protects early records against an adversary that induces with current material, and considerably less against one that does not. Little archival knowledge is needed (registered): with a tenth of each topic's induction budget drawn from cohort H's era and the rest current, the advantage moves from −0.019 to +0.103, and with a quarter to +0.216, against +0.405 for a fully archival adversary. The coarse-taxonomy arm has chance one in four rather than one in twelve. At the largest gap the advantage is below zero, because a map fitted on the induced era mislabels

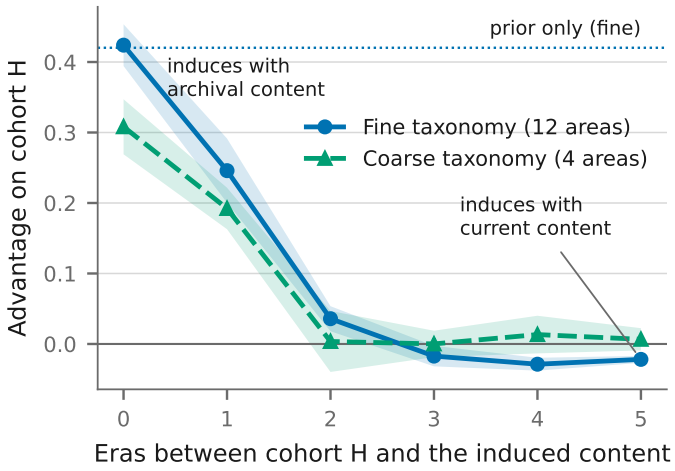


Fig. 3. What subject drift is actually worth. The horizontal axis is the time gap between cohort H and the content the adversary induces with; bands are 95% Student-t intervals over five k-means seeds. An adversary that supplies archival content from cohort H's own era recovers the advantage drift appears to remove; in raw accuracy that matches always guessing cohort H's most frequent topic (dotted line), and in balanced accuracy it is well above chance (supplementary material).

a drifted cohort more often than a guess (supplementary material). Cohort H here is skewed (its commonest topic is 0.50 of it), so raw accuracy flatters any adversary that leans on the prior. Balanced accuracy, the mean per-topic recall, does not: in it the archival adversary's advantage is 0.217 [0.151, 0.283] and the current-content one's 0.055 [0.027, 0.082] (not registered), so the labels carry information about individual records, and drift erodes it.

7.2 Deployed Stores, and Time-Placed Controls

The attack was run against four deployed stores (Chroma, FAISS, LanceDB and sqlite-vec), an indexed LanceDB variant and an append-only control, each inside a `gocryptfs` volume, whose ciphertext mirrors the plaintext file structure block for block. Block labels are fitted on one half of the induced writes and scored on the other, and induced writes are interleaved across topics so that tail-appending stores cannot make write order masquerade as content placement.

Transfer was not detected in any of the six configurations at this budget: held-out localisation sits at chance, and no interval on $\mathrm{Adv}_{\mathrm{hist}}$ reaches above $+0.004$ over 10 seeds. In the time-placed stores, and in those that rewrite their whole index on every save, an induced write touches none of the blocks that hold cohort-H records. Chroma's HNSW insert does touch cohort-H blocks, yet the labels carry no topic, $\mathrm{Adv}_{\mathrm{hist}} = -0.011$ $[-0.025, +0.004]$, because which blocks an insert touches depends on graph structure and insertion order. These stores establish the premise of Lemma 1, not its conclusion: transfer needs a layout that discloses which partition holds a record at the observer's granularity.

A deployed store the attack reaches: pgvector: PostgreSQL's `pgvector` with an `ivfflat` index, built on the cohort and inserted into afterward through real SQL with a checkpoint after each write, in the same `gocryptfs` volume, assigns an insert to its nearest list and writes that list's pages. The attack reaches cohort H: held-out accuracy on it is 0.243 against chance 0.125, an advantage of 0.118 [0.094, 0.143] over 20 seeds, the interval above zero (registered before the run). It is weaker than the growing layout, because write-ahead-log and heap churn blur list pages at 4 KiB, but it is a deployed, content-placed store whose placement transfers to the pre-compromise cohort. A checkpoint after every write is the finest capture an adversary can hope for. Capturing one version per g writes instead (registered, 10 seeds per point, twice the induction budget), the advantage is 0.187 [0.155, 0.219] at $g = 1$ and still 0.067 [0.043, 0.092] at $g = 4$; at $g = 16$ every version mixes all eight topics, the estimator labels no block, and the advantage is exactly zero. Coarse capture is a real defence against this attack, and Section 7.7 measures what a real synchronization client provides. Structure transfer is therefore neither universal nor hypothetical: most deployed stores tested do not expose a content-disclosing layout at block granularity, and one widely used store does.

7.3 Real-Agent Induction and the Deduplication Oracle

The attack runs equally against memories a local model authored from adversary-supplied issues (supplementary material), across burst sizes: a writing turn yields several memories on one subject within milliseconds, so a realistic flush carries a small same-topic burst. The growing layout is the positive control: labels learned from these memories transfer to the earlier cohort at every budget, so agent-written text keeps the structure the attack needs. Against Mnemos-P the harness supplies cohort-H membership, as a version-diff adversary would obtain it, isolating what padding does to labelling. A single-record attack stays near chance, no interval above $+0.017$, but a burst multiplies the signal against the fixed cover: at four records per turn the mean advantage is 9.9 times the single-record value, peaking at $+0.062$ $[0.051, 0.073]$. We registered that the padded design would stay within 0.05 of chance throughout; it does not.

The attack is cheap: on the growing layout the advantage clears zero at 40 supplied messages over ten topics, about 3 min of agent time.

The deduplication oracle of Section 4.5 observes only *whether a write happened* after the adversary supplies a document. With a similarity threshold deciding, the oracle is perfect; with a model deciding, as in mem0's pipeline, `qwen2.5:7b-instruct` writes 0.74 of facts the store already holds against 0.97 of new ones, an AUC of 0.62 [0.57, 0.67] against Mnemos-P. Section 7.5 measures the oracle in unmodified libraries, and its closure.

7.4 The Dual-Epoch Game

Table 3 plays the game of Definition 1 against the compiled engines, with admissibility *verified* (every admissible arm asserts a zero refusal counter), held-out evaluation, and a hand-built statistic beside the learned classifier in every arm. Removing the keyed permutation, which Theorem 1 does not require, leaves the game at chance once histories are genuinely admissible; what made an identity permutation look distinguishable was the refusal channel. Removing S1 or S2 is perfectly distinguishable. The inadmissible arms show the refusal channel and its closure by padded refusal.

Three arms test Lemma 2, registered before they ran. Writing only the placed slot, drawn uniformly, is at chance,

TABLE 3
The dual-epoch game against the compiled engines, 1000 instances per bit, evaluated on a held-out half. *Learned* is a gradient-boosted classifier over snapshot and trace features; *hand* is one named statistic per arm, oriented on the training half. A test bounds leakage from below, so the upper end of the interval is the ceiling for these attackers and not a certificate.

Configuration	Admissible	Refusals	Learned AUC [95% CI]	Hand AUC [95% CI]	Verdict
P shipped	yes	0	0.50 [0.47, 0.54]	0.52 [0.48, 0.56]	at chance
P no S1 (growing regions)	yes	0	1.00 [1.00, 1.00]	1.00 [1.00, 1.00]	distinguishable
P no S2 (unsealed slots)	yes	0	1.00 [1.00, 1.00]	1.00 [0.99, 1.00]	distinguishable
P no S3 (per-slot writes)	yes	0	0.49 [0.45, 0.53]	0.50 [0.46, 0.53]	at chance
P identity permutation	yes	0	0.52 [0.49, 0.56]	0.51 [0.47, 0.54]	at chance
P inadmissible, padded refusal	**no**	7659	0.47 [0.44, 0.51]	0.53 [0.49, 0.56]	at chance
P inadmissible, silent refusal	**no**	7659	1.00 [1.00, 1.00]	0.99 [0.99, 0.99]	distinguishable
M shipped	yes	0	0.50 [0.47, 0.54]	0.51 [0.47, 0.55]	at chance
M content placement	yes	0	0.49 [0.45, 0.52]	0.49 [0.45, 0.52]	at chance
P first-free slot (S3 kept)	yes	0	0.51 [0.48, 0.55]	0.50 [0.50, 0.50]	at chance
P no S3, first-free slot	yes	0	1.00 [1.00, 1.00]	1.00 [1.00, 1.00]	distinguishable

0.49 [0.45, 0.53]; so is whole-region rewriting with lowest-free-slot placement, 0.51 [0.48, 0.55]. Removing both, so the index of the one slot written is the region's load, gives 1.00 [1.00, 1.00]. Either protection suffices, so S3 is a condition on the pair (placement rule, write granularity).

7.5 The Reactive Game

In the game of Definition 1 Mnemos-M and a variant of it that places by content are both at chance, because the adversary supplies every operation. The reactive game separates them. We run it with a deduplicating agent that writes a supplied fact only if no near-duplicate is stored, and report the held-out AUC of the strongest trace statistic per design. The registered prediction was that only the clock-only arms sit at chance.

The shipped memory-resident store is at chance, 0.505 [0.475, 0.538], and so is its whole-region-flush variant, 0.490 [0.456, 0.523]. Flushing when the agent writes is fully distinguished, 1.000 [1.000, 1.000]: the occurrence of a flush is the deduplication bit. Placing by content under a clock is distinguished too, 0.976 [0.951, 0.995], because a reactive write lands in the retrieved fact's region. Mnemos-P, dual-epoch secure by Theorem 1, is fully distinguished, 1.000 [1.000, 1.000]. This is the empirical form of DE $\not\Rightarrow$ RDE, and the reason the positive result for Mnemos-M is Theorem 2 rather than a row of Table 3.

In unmodified libraries: Table 4 plays the same game with real agent-memory libraries, unmodified, the adversary supplying a fact and observing only the storage directory (registered). Whether the oracle exists is a question about the writer, as Theorem 2 says it must be. mem0 drops an extracted fact whose hash it already holds, so the oracle is open exactly when the deciding model restates a stored fact in the words it was stored in: `qwen2.5` and `mistral` do so on about half of member probes and the oracle is open, `gemma2` on about a quarter and it is open less widely, and `llama3.1` rewords nearly every one, a write occurs either way, and the oracle is closed. In all four, the oracle's strength follows the gap between the member and non-member write rates. LangMem's manager re-stores the memories it retrieves and patches on every call, so it too writes regardless of membership. Re-probing a quiet fact with paraphrases adds nothing for `qwen2.5` (AUC 0.742 to 0.745): a paraphrase is a new string to mem0's hash check, so it writes whether or not the fact is held.

TABLE 4
The deduplication oracle in unmodified libraries. *Top*: who decides. mem0 2.2.1 on its shipped Qdrant store with four local models making the add/skip decision, and LangMem 0.0.30's store manager; columns give the rate at which a probe wrote for a fact already stored (member) and a novel one (non-member). *Bottom*: what the store reveals. mem0 with `qwen2.5`, its vector store replaced through mem0's own interface; columns give the mean 4 KiB blocks changed in a fixed 16 s window. AUC is for "non-member", oriented in advance, with a bootstrap interval over 60 probes per class.

Library, store	Deciding model	Member	Non-mem.	AUC [95% CI]
mem0, Qdrant	`qwen2.5:7b`	0.47	0.95	0.74 [0.67, 0.81]
mem0, Qdrant	`llama3.1:8b`	0.98	0.93	0.48 [0.44, 0.51]
mem0, Qdrant	`mistral:7b`	0.43	1.00	0.78 [0.72, 0.84]
mem0, Qdrant	`gemma2:9b`	0.72	0.97	0.62 [0.56, 0.69]
LangMem, file store	`qwen2.5:7b`	1.00	1.00	0.50 [0.50, 0.50]
mem0, Qdrant	`qwen2.5:7b`	2	7	0.83 [0.76, 0.90]
mem0, Mnemos-P	`qwen2.5:7b`	831	1528	0.69 [0.60, 0.78]
mem0, Mnemos-M	`qwen2.5:7b`	652	652	0.47 [0.37, 0.56]
+ history file	`qwen2.5:7b`	540	539	0.63 [0.52, 0.73]

The store is the other half of the system, and the bottom rows replace only mem0's vector store, through its own `VectorStoreBase` interface. Shipped mem0 leaks membership at 0.83 [0.76, 0.90]. Mnemos-P, dual-epoch secure, still leaks at 0.69 [0.60, 0.78], because it flushes when mem0 writes. Mnemos-M, unmodified mem0 above it, sits at chance, 0.47 [0.37, 0.56]: its sweep changes the same number of blocks whether or not mem0 inserted anything. The scope condition of Theorem 2 is visible too: an adversary that also sees mem0's own SQLite history file, which mem0 appends to on every insertion, regains part of the signal, 0.63 [0.52, 0.73], so a deployment keeps that file in the store or outside the synchronized scope.

7.6 The Byte-Level Auditor

An auditor diffs the store byte by byte around every operation of both sealed engines and requires each changed range to be named by an emitted event and each event to match a real change. For Mnemos-P it confirms that a flush rewrites exactly $w + 1$ whole regions, that searches, deletes and sub-batch puts write nothing until the next flush, that a refused put still rewrites its region, and that

every byte of a rewritten region changes. For Mnemos-M it confirms that a search writes and reads nothing, and that a sweep flush writes exactly the cursor's slot run and the journal, the same byte ranges whether or not a record was carried. Each check fails under the ablation that removes its mechanism. The auditor also found a defect no functional test can: rewrites owed to refused writes were queued behind accepted ones, so a refusal-heavy history built a backlog whose length distinguished it.

7.7 Channels That Reach the Earlier Cohort

Theorem 1 closes the snapshot channel. Three channels reach cohort H anyway.

The write channel: Frequency analysis labels regions with the topics of *post-compromise* writes. Against a growing layout the labels transfer to the earlier cohort; against a sealed layout they apply to a cohort-H record only if another channel supplies its region (supplementary material). The two adversary classes separate here too: against Mnemos-P a read observer recovers the keyed permutation from probe co-occurrence (Section 7.8) and labels partitions from public text with no induced writes, and a write observer does neither. Against Mnemos-M neither learns anything.

Multiple versions: Version diffs supply that membership. The region holding a pre-τ record is inside the diff at every granularity tested; granularity sets how precisely it is named, down to the $1/B$ null when the diff covers the whole file. Re-encrypting the whole file at a session boundary makes a cross-session diff total and drives the precision to exactly the null. A real backup tool keeps versions the attack can use (registered): backing Mnemos-P up with restic after every flush and restoring each snapshot through restic's own interface gives precision 0.207 [0.205, 0.209] against the $1/64$ null (13 times the null rate), and 0.058 [0.057, 0.058] at one snapshot per four flushes. An adversary holding the encrypted repository without its key sees only how much each snapshot added; for Mnemos-M that is exactly 1.0 new chunk per snapshot, a constant. A synchronization client chooses its own schedule (registered): with two Syncthing 2.1.5 instances, staggered versioning on the receiving side and an agent writing every 30 s, the client retained one version per 3.0 flushes, and the diff of the retained versions around a record named its region at 4.7 times the null rate, the region among the candidates every time. At one write every 5 s the client keeps one version per 13.9 flushes and the ratio falls to 1.4: the client's schedule relative to the agent's sets the capture granularity of Section 3.3.

The composed attack: We ran the full chain, version diffs to membership, induced writes to labels, composed to a cohort-H topic, on Mnemos-P at $g \in \{1, 2, 4, 16\}$. The registered prediction, that the composition would reach cohort H, *failed*. On agent memories the induced-label map does not localize topics on the sealed, padded layout: even an oracle handed each cohort-H record's exact region scores -0.003, consistent with the Mnemos-P rows of the agent-written attack, and the composed attack, choosing among the 4.8 candidate regions a $g = 1$ diff names, scores -0.035. The multi-version channel is real, but on this corpus it does not compose into topic recovery, because the label map it would feed is weak on the partitioned design.

Refusals: An adversary that induces writes into a region and watches whether it is rewritten measures Proposition 5 directly. A refusal that writes nothing rewrites a full region on 0.070 of induced writes against 1.000 for an open one, an AUC of 1.000 [1.000, 1.000]; padded refusal closes it, 0.500 [0.500, 0.500]. Under a skewed corpus the partitioned design refuses its first write at 23.8% of nominal capacity, while Mnemos-M, with only a global bound, reaches 100.0%. Sweeping $B \in \{64, 128, 256\}$, $C \in \{16, 32, 64\}$ and the Zipf skew of the topic workload (registered), Mnemos-P's first refusal comes at 6% to 24% of nominal capacity, earlier as B grows and as the skew rises (9% to 6% from Zipf 0.8 to 1.2), because the first partition to fill is the most frequent topic's; Mnemos-M's never comes before 100%. A partitioned store that must keep every write is provisioned for its most frequent topic; padded refusal trades that provisioning for lost writes, a cost in availability, not confidentiality.

7.8 Read-Side Channels, for the RW Class Only

These results apply to the read-write observer, not to the write observer of the anchoring instantiations, which sees no reads.

Search and deletion: The record-store design issues 2,000 result-dependent reads over a search trace, attaching a query topic to 292 record positions, and names the region of a deleted cohort-H record immediately in 99 of 100 deletions. The partitioned design issues 0 such reads and names 0 regions; Mnemos-M's search does not touch the disk.

Query equality: We measure it over 20 independent stores, fixing the adversary's overlap threshold on half the queries and scoring it on the rest. The shipped randomized superset defeats a test for exact equality of read sets (Youden's $J = 0.00$). It does not defeat a threshold on their overlap ($J = 0.98$ $[0.97, 0.99]$), because repeated searches share every genuine probe. Grouped probing leaks pairwise equality at group granularity ($J = 0.85$ $[0.82, 0.88]$). Sweeping each knob traces the frontier (registered, 20 stores per point). The spread leaks $J = 0.966$ $[0.932, 0.991]$ at $s = 1$, 0.574 $[0.558, 0.590]$ at $s = 4$ and 0.186 $[0.173, 0.197]$ at $s = 8$, where it issues eight times the genuine reads. Grouping leaks 0.959 $[0.943, 0.973]$ at group size 2 and 0.459 $[0.393, 0.509]$ at 32, which costs $8.0\times$ the reads. Neither dominates. At twice the reads grouping leaks less (0.834 $[0.800, 0.866]$ against 0.978 $[0.970, 0.985]$); from four times upward the spread leaks less. No point reaches chance, so closing the channel requires oblivious access.

Permutation recovery: Genuine probes are the nearest partitions to a query, so regions read together are geometric neighbours, and matching the observed co-occurrence graph to the one the public table predicts recovers the permutation from ordinary search traffic, with no induced writes. Without probe randomization, an approximate quadratic-assignment solver refined by pairwise swaps places 0.963 [0.897, 1.000] of the B regions exactly over 20 independent stores (the solver alone reaches 0.887, so the number is a lower bound). The randomized superset degrades this to 0.564, and grouped probing defeats it, 0.025 against a $1/B$ null of 0.016. The permutation protects against a write observer holding a public table, not against a read observer.

7.9 Cost

We measure the three designs against the in-tree Rust plaintext IVF, which shares `QuantVec` and the partitioner with the encrypted engines, at five sizes. Writes are timed in the sparse regime, where an agent writing seconds apart pays a whole flush per record; the supplementary material plots every measure against n. Exhaustive in-memory ranking is 9.1 times faster than the partitioned design at 1,000 records but only 0.29 times as fast at 100,000. Because it is linear in n, it crosses the partitioned design between 10,000 and 30,000 records. Probing logical partitions in memory removes this inversion without reopening the write channel. At eight partitions scored per query, Mnemos-M with its in-memory index is 3.8 to 18.3 times faster than Mnemos-P (1.78 ms against 6.67 ms at the largest size), because it reads and decrypts nothing. It pays instead at unlock (6.7 s against 0.8 s at 100,000 records) and in resident memory (544 MB). A sparse write costs 32.2 ms against 46.3 ms. The supplementary material tabulates every size. It also prices fetching results once reads are visible: by position, by full scan, and with Path ORAM as a stated model.

What the clocked flush writes: A clocked flush is paid whether or not the agent wrote. Rewriting $w + 1$ whole regions each tick is 5.2 MiB a flush at the shipped geometry, 945 GB a day at a 500 ms clock. The sweep flush, in the manner of write-only ORAM [45]–[47], rewrites the next fixed run of slots after a cursor plus a fixed-size sealed journal that holds staged records until the cursor reaches a free slot. It writes 68.4 KiB per flush whether the flush carries a record or not, and at $\Delta = 10$ s, 0.60 GB a day, roughly 1,562$\times$ less. Locking Mnemos-M rewrites nothing extra; locking Mnemos-P rewrites its whole file, 78 MiB, which a synchronization client uploads in full each session. A chunking backup erodes the sweep's upload advantage: a restic snapshot of Mnemos-M adds 959 KiB for 9 KiB rewritten, and of Mnemos-P 1099 KiB for 85 KiB.

Above the memory limit: Against W the partitioned design is not the right tool above the memory limit: Mnemos-M/disk's writes remain the sweep's, so every write-side channel stays closed at any size. At 10^5 records (registered) it holds 0.6 MB resident against Mnemos-M's 544.3 MB. It searches in 13.5 ms against Mnemos-P's 6.5 ms and writes a sparse record in 1.6 ms against 42.4 ms. It pays 8.2 s at unlock to read every header. Resident memory and unlock time are linear in n, so the measurement extrapolates to a few megabytes and about a minute and a half at 10^6; we did not measure beyond 10^5. A read observer sees its search access pattern, which only oblivious search closes [50]–[52]. The partitioned design remains the construction Theorem 1 is proved for, not the recommended deployment.

7.10 Retrieval Quality

Retrieval is measured against BEIR relevance judgments on the *full* corpora (truncating FiQA to 5,000 documents, under a tenth of its 57,638, inflates plaintext nDCG@10 well above 0.384), against a plaintext IVF at the same probe width, and counting capacity refusals during indexing (0). Quantisation to int8 costs at most 0.001 nDCG@10 on any domain, and fixed probing up to 0.080. Mnemos-M's in-memory index, scoring the same eight partitions as Mnemos-P at $s = 2$, gains +0.023 to +0.045 on every domain, since all eight are the query's nearest where half of Mnemos-P's are cover. Both findings hold with a second, independently trained embedding model, `nomic-embed-text` (registered): int8 costs -0.000 on SciFact and -0.000 on NFCorpus, and the in-memory index scores 0.652 and 0.318 against Mnemos-P's 0.604 and 0.291.

Grouped probing, the one mechanism that resists permutation recovery, costs retrieval: nDCG@10 falls from 0.658 to 0.435 on SciFact and from 0.329 to 0.224 on FiQA against the $s = 2$ spread, which is why we recommend the design that keeps search off the disk.

Condition S4 has a price of its own, and it is paid in capacity rather than ranking (registered; the prediction of a gain under 0.15 failed). A public table, trained on a different corpus, ranks SciFact about as well as one trained on SciFact itself (plaintext IVF 0.672 against 0.657), but it puts 9.9% of the corpus in one partition, so Mnemos-P refuses 2,297 documents while indexing and scores 0.441 against 0.665. Mnemos-M, with no per-partition bound, scores 0.708 and 0.703: its logical table can be public at no cost, and could be retrained in memory without a disk write.

7.11 Operating-System Artifacts

The audit of Section 3.4 is run, not asserted: on the evaluation machine it recovers change-journal records naming the store's files with timestamps, enumerates shadow copies and synchronized trees, reports the hibernation file, and verifies that a locked engine refuses to search. Graded on a ladder (exists; relevant; obtainable; exploitable for the historical attack; sufficient alone), the change journal reaches rung 2, since it dates writes but gives no region diff; shadow copies and file-history snapshots reach rung 4, giving the multi-version channel at the snapshot schedule's granularity; retained flash pages reach rung 4 for Mnemos-P, whose rewrites land at fixed offsets, and rung 1 for Mnemos-M; and the hibernation image, against key custody rather than the layout, is the one rung-5 artifact. The study is Windows-specific; the Linux counterparts occupy the same rungs but are not measured.

7.12 Where the Information Goes

The supplementary material puts every channel on one axis in a leakage ledger. Where an experiment recorded a held-out accuracy, Fano's inequality with the secret's empirical entropy turns it into a lower bound on the mutual information between the secret and the adversary's view; an AUC is reported as an AUC and is not converted. At a spread of one, permutation recovery carries at least 4.9 of the $\log_2 B = 6$ bits of each region's partition, and grouped probing takes it to zero; the multi-version channel depends on capture granularity and vanishes across a session once the file is re-encrypted at lock. For a row where nothing was detected, the informative number is the last column, the smallest effect the experiment had 80% power to detect: "not detected" is a bounded statement about this attacker at this sample size, not a certificate.

TABLE 5
Where each scheme leaks about the pre-compromise cohort, grouped by adversary class. *Open*: the channel carries pre-compromise information. *closed*: it does not. *n/a*: the mechanism that would carry it is absent. *not det.*: no transfer was detected at the granularity tested, which is weaker than closed (Section 7.2). The W block holds the write-observing channels; the RW block adds the read-side ones, which apply only to a read observer. Rows above the rule are constructions this paper attacks or compares against; rows below are the designs it evaluates.

	Class W (write observer)						Class RW (adds reads)	
Scheme	Snapshot	Write	Refusal	Multi-ver.	Derived	Delete	Search	Query eq.
Growing encrypted IVF	open	open	n/a	open	open	open	open	open
pgvector `ivfflat`, `gocryptfs`	open	open	n/a	open	open	open	open	open
Other deployed stores, `gocryptfs`	not det.	not det.	n/a	not det.	not det.	not det.	not det.	not det.
Forward-private SSE, sealed+padded	open (ctrs)	closed	n/a	open	open	graded	open	open
Write-only ORAM index	closed	closed	n/a	closed	closed	closed	open	open
ORAM over the slots	closed	closed	n/a	closed	closed	closed	closed	closed
Mnemos-RS	open	open	open	open	open	open	open	open
Mnemos-P	closed	open	closed†	closed‡	open	closed	closed	open
Mnemos-M/disk	closed	closed	n/a	closed	closed§	closed	open	open
Mnemos-M	closed	closed	n/a	closed	closed§	closed	n/a	n/a

† by performing the rewrites an accepted write would have (Proposition 5). ‡ closed across session boundaries by re-encryption at lock; within a session, version diffs name a record's region (Section 7.7). § given the clocked flush schedule; the clock itself remains visible. Mnemos-M/disk keeps headers on disk for stores larger than memory: its writes stay clock-driven and invisible to W, so it matches the write-only ORAM row, while a read observer sees its search access pattern.

8 Discussion

8.1 Positioning

Table 5 places the designs among the alternatives. Against the write observer W, a write-only oblivious index closes every channel, and Mnemos-M/disk matches it by writing on a clock; Mnemos-M also removes the disk from the query path, which makes search and query equality *absent* rather than oblivious. Against the read observer RW only full obliviousness closes search and query equality, at a per-access cost: the ORAM-based systems (compared in the supplementary material) are the reference for that stronger adversary, not a baseline this work claims to beat, and they buy read-side obliviousness a write observer does not need.

8.2 Residual Leakage

Mnemos-P's residuals are those of Table 5: an open write channel, query equality, a permutation a read observer recovers, version diffs closed only across sessions, and derived writes. Mnemos-M's clock is visible, so an observer learns when the store was open though not whether anything was written; without back-pressure, a lock after a burst the sweep could not place reveals that the backlog exceeded the journal. Back-pressure removes that from the disk, not from the agent: a refused put waits a tick, so an adversary who induces more than J writes and can time the agent's reply learns that the journal was full. C1 hides activity from a storage observer ($\mathcal{L}_{\text{clock}}$); against a timing observer J must exceed the largest per-turn burst. The guarantees cover the store's own files: an agent framework that persists other state whose writes follow the agent's decisions must keep it in the store or outside the observed scope (Theorem 2). Both Mnemos-M variants inherit A3: if the adversary can read process memory, a memory-resident index is the worst place to have put the plaintext, which is why hibernation is a deployment requirement. No storage-layer measure reaches copies a synchronization provider has already retained.

8.3 Limitations

Assumptions: The analysis depends on A2, a clean host before τ, and A3, that the adversary cannot read process memory. Decrypted results and the embedding computation reside in process memory; the locked region holds plaintext during an operation, and for the memory-resident design for the whole session, but the Python embedder sits outside it, so plaintext embeddings may reach swap on the embedding path. Embeddings invert to text and an imaging adversary obtains the pagefile (Section 7.11), so a deployment against it should run embedding inference inside the locked arena (a quantized model from Rust or ONNX) or require encrypted or disabled swap.

Agent experiments: The agent results use local models of moderate size, and the induced corpus is issue text from one public tracker. The deduplication oracle is measured end to end in mem0 and LangMem with four local models of 7–9B parameters; hosted frontier models and the remaining frameworks the census flags were not run, and whether a given model opens the oracle has to be measured, not assumed.

Scope of the measurements: Performance is measured on one machine (Intel i9-14900HX, 32 GB, NVMe, Windows 11, with WSL2 for the stacked encrypted file system). The deployed-store experiments use local instances holding public data under one encryption layer, over 10 store builds per store (20 for pgvector), and their per-store conclusions depend on library versions. The version-granularity parameter is swept rather than measured against a provider's retention policy. Performance is measured to 10^5 records, and retrieval with a second embedding model on two domains only. Key custody is implemented for Windows only.

Construction: Both sealed designs remove a tombstoned record only when the maintenance cursor reaches its region. Records larger than one payload block are rejected rather than chunked. The partition table is fixed at creation, so a store whose subject matter drifts far enough eventually

partitions poorly. Rebuilding it is not evaluated: for Mnemos-P a rebuild rewrites every region and a corpus-trained table would violate S4, while Mnemos-M's logical table could be retrained in memory without a disk write.

8.4 Ethical Considerations

The attacks were run against implementations built for this study and against public or synthetic corpora; no third-party production system was attacked, and the deployed stores and library of Sections 7.2 and 7.3 were local instances holding public data. The countermeasures are layout changes designers of encrypted indexes can adopt, and are disclosed with the attacks.

8.5 Disclosure of AI Assistance

The implementation and manuscript were prepared with the assistance of an AI coding tool used under the author's direction and review. All experiments, measurements and claims were executed and verified by the author, and the code and data are available so that every reported number can be reproduced independently.

9 Conclusion

Historical confidentiality, the protection of what an agent remembered before its storage was compromised, is a property of its own: encryption, snapshot security and history independence do not deliver it. For content-placed indexes, induced writes label partitions with topics, and where the layout discloses which partition holds each stored record those labels apply to records written before the compromise; the transfer lemma makes that condition precise, and a deployed store, pgvector under a stacked encrypted file system, meets it.

Fixed-size sealed regions close the snapshot channel, overflow included, and three channels reach the earlier cohort anyway: retained versions, refusals, and the agent's own operations, down to the occurrence of a write, which makes deduplication a membership oracle in a widely used library whenever its model writes conditionally. Against a self-writing agent a store is secure exactly when what it reveals is independent of what the agent does. A store whose disk behaviour is a function of the clock alone meets that condition: placement independent of content, search in locked memory over a logical index, and a clocked write-only-oblivious sweep, at 68.4 KiB per flush. Keeping headers on disk extends it past the memory limit with its writes still invisible to a write observer.

Two problems remain open: a read observer of a store too large to keep resident is defeated only by oblivious search, whose cost against these designs is unmeasured, and version retention at a provider is beyond the reach of the storage layer.

Availability

The three designs, the attack harness, the byte-level auditor, and the corpora and scripts that regenerate every table and number from the stored results are at https://github.com/shreyasfegade/mnemos, to be archived with a persistent identifier on publication. Full proofs are in the supplementary material.

References

[1] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," arXiv:2310.08560, 2023.

[2] S. Dong, S. Xu, P. He, Y. Li, J. Tang, T. Liu, H. Liu, and Z. Xiang, "Memory injection attacks on LLM agents via query-only interaction," arXiv:2503.03704, 2025.

[3] Z. Chen, Z. Xiang, C. Xiao, D. Song, and B. Li, "AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2024.

[4] S. S. Srivastava and H. He, "MemoryGraft: Persistent compromise of LLM agents via poisoned experience retrieval," arXiv:2512.16962, 2025.

[5] OWASP Foundation, "LLM08:2025 vector and embedding weaknesses," OWASP Top 10 for Large Language Model Applications, 2025.

[6] B. Wang, W. He, S. Zeng, Z. Xiang, Y. Xing, J. Tang, and P. He, "Unveiling privacy risks in LLM agent memory," in *Proc. Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2025.

[7] K. Chen, Y. Pang, and T. Wang, "MRMMIA: Membership inference attacks on memory in chat agents," *arXiv preprint arXiv:2605.27825*, 2026.

[8] J. X. Morris, V. Kuleshov, V. Shmatikov, and A. M. Rush, "Text embeddings reveal (almost) as much as text," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2023, pp. 12 448–12 460.

[9] M. Naor and V. Teague, "Anti-persistence: History independent data structures," in *Proc. ACM Symp. Theory Comput. (STOC)*, 2001.

[10] R. Curtmola, J. Garay, S. Kamara, and R. Ostrovsky, "Searchable symmetric encryption: Improved definitions and efficient constructions," in *Proc. ACM Conf. Comput. Commun. Secur. (CCS)*, 2006, pp. 79–88.

[11] S. Kamara, T. Moataz, and O. Ohrimenko, "Structured encryption and leakage suppression," in *Advances in Cryptology – CRYPTO 2018*, 2018.

[12] M. S. Islam, M. Kuzu, and M. Kantarcioglu, "Access pattern disclosure on searchable encryption: Ramification, attack and mitigation," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2012.

[13] D. Cash, P. Grubbs, J. Perry, and T. Ristenpart, "Leakage-abuse attacks against searchable encryption," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2015.

[14] L. Blackstone, S. Kamara, and T. Moataz, "Revisiting leakage abuse attacks," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2020.

[15] G. Kellaris, G. Kollios, K. Nissim, and A. O'Neill, "Generic attacks on secure outsourced databases," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2016, pp. 1329–1340.

[16] P. Grubbs, M.-S. Lacharité, B. Minaud, and K. G. Paterson, "Pump up the volume: Practical database reconstruction from volume leakage on range queries," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2018.

[17] S. Oya and F. Kerschbaum, "IHOP: Improved statistical query recovery against searchable symmetric encryption through quadratic optimization," in *Proc. USENIX Secur. Symp.*, 2022, pp. 2407–2424.

[18] E. M. Kornaropoulos, C. Papamanthou, and R. Tamassia, "Data recovery on encrypted databases with k-nearest neighbor query leakage," in *IEEE Symposium on Security and Privacy (S&P)*, 2019.

[19] S. Kamara and T. Moataz, "Computationally volume-hiding structured encryption," in *Advances in Cryptology – EUROCRYPT 2019*, 2019, pp. 183–213.

[20] S. Patel, G. Persiano, K. Yeo, and M. Yung, "Mitigating leakage in secure cloud-hosted data structures: Volume-hiding for multi-maps via hashing," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2019, pp. 79–93.

[21] L. Xu, L. Zheng, C. Xu, X. Yuan, and C. Wang, "Leakage-abuse attacks against forward and backward private searchable symmetric encryption," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2023.

[22] B. Minaud and M. Reichle, "Dynamic local searchable symmetric encryption," in *Advances in Cryptology — CRYPTO*, 2022.

[23] E. Stefanov, C. Papamanthou, and E. Shi, "Practical dynamic searchable encryption with small leakage," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2014.

[24] R. Bost, "$\Sigma o \varphi o \varsigma$: Forward secure searchable encryption," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2016.

[25] Y. Zhang, J. Katz, and C. Papamanthou, "All your queries are belong to us: The power of file-injection attacks on searchable encryption," in *Proc. USENIX Secur. Symp.*, 2016.

[26] R. Bost, B. Minaud, and O. Ohrimenko, "Forward and backward private searchable encryption from constrained cryptographic primitives," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2017.

[27] J. Ghareh Chamani, D. Papadopoulos, C. Papamanthou, and R. Jalili, "New constructions for forward and backward private symmetric searchable encryption," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2018.

[28] G. Amjad, S. Kamara, and T. Moataz, "Injection-secure structured and searchable symmetric encryption," in *Advances in Cryptology – ASIACRYPT 2023*, 2023.

[29] B. Fuller, M. Varia, A. Yerukhimovich, E. Shen, A. Hamlin, V. Gadepally, R. Shay, J. D. Mitchell, and R. K. Cunningham, "SoK: Cryptographically protected database search," in *Proc. IEEE Symp. Secur. Privacy (S&P)*, 2017.

[30] G. Amjad, S. Kamara, and T. Moataz, "Breach-resistant structured encryption," *Proc. Privacy Enhancing Technol. (PoPETs)*, 2019.

[31] Y. Du, D. Genkin, and P. Grubbs, "Snapshot-oblivious RAMs: Sub-logarithmic efficiency for short transcripts," in *Advances in Cryptology – CRYPTO 2022*, 2022.

[32] G. Persiano and K. Yeo, "Limits of breach-resistant and snapshot-oblivious RAMs," in *Advances in Cryptology – CRYPTO 2023*, 2023.

[33] P. Grubbs, T. Ristenpart, and V. Shmatikov, "Why your encrypted database is not secure," in *Proc. Workshop Hot Topics Operating Syst. (HotOS)*, 2017.

[34] P. Grubbs, R. McPherson, M. Naveed, T. Ristenpart, and V. Shmatikov, "Breaking web applications built on top of encrypted data," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2016.

[35] A. Czeskis, D. J. St. Hilaire, K. Koscher, S. D. Gribble, T. Kohno, and B. Schneier, "Defeating encrypted and deniable file systems: TrueCrypt v5.1a and the case of the tattling OS and applications," in *Proc. USENIX Workshop on Hot Topics in Security (HotSec)*, 2008.

[36] T. M. Peters, M. A. Gondree, and Z. N. J. Peterson, "DEFY: A deniable, encrypted file system for log-structured storage," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2015.

[37] M. Wei, L. M. Grupp, F. E. Spada, and S. Swanson, "Reliably erasing data from flash-based solid state drives," in *USENIX Conference on File and Storage Technologies (FAST)*, 2011.

[38] J. Reardon, D. Basin, and S. Capkun, "SoK: Secure data deletion," in *Proc. IEEE Symp. Secur. Privacy (S&P)*, 2013, pp. 301–315.

[39] J. Hofmann and K. T. Truong, "End-to-end encrypted cloud storage in the wild: A broken ecosystem," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2024.

[40] D. Micciancio, "Oblivious data structures: Applications to cryptography," in *Proc. ACM Symp. Theory Comput. (STOC)*, 1997, pp. 456–464.

[41] N. Buchbinder and E. Petrank, "Lower and upper bounds on obtaining history independence," *Information and Computation*, vol. 204, no. 2, pp. 291–337, 2006.

[42] S. Bajaj and R. Sion, "HIFS: History independence for file systems," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2013, pp. 1285–1296.

[43] D. S. Roche, A. J. Aviv, and S. G. Choi, "A practical oblivious map data structure with secure deletion and history independence," in *Proc. IEEE Symp. Secur. Privacy (S&P)*, 2016.

[44] M. Trabucco, M. Andreolini, and L. Ferretti, "Practical and secure history-independent indexing for queryable-encrypted databases," *J. Inf. Secur. Appl.*, vol. 89, p. 104439, 2026.

[45] E.-O. Blass, T. Mayberry, G. Noubir, and K. Onarlioglu, "Toward robust hidden volumes using write-only oblivious RAM," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2014.

[46] D. S. Roche, A. J. Aviv, S. G. Choi, and T. Mayberry, "Deterministic, stash-free write-only ORAM," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2017.

[47] A. J. Aviv, S. G. Choi, T. Mayberry, and D. S. Roche, "ObliviSync: Practical oblivious file backup and synchronization," in *Proc. Network and Distributed System Security Symp. (NDSS)*, 2017.

[48] O. Goldreich and R. Ostrovsky, "Software protection and simulation on oblivious RAMs," *J. ACM*, vol. 43, no. 3, pp. 431–473, 1996.

[49] E. Stefanov, M. van Dijk, E. Shi, C. Fletcher, L. Ren, X. Yu, and S. Devadas, "Path ORAM: An extremely simple oblivious RAM protocol," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2013.

[50] J. Zhu, L. Patel, M. Zaharia, and R. A. Popa, "Compass: Encrypted semantic search with high accuracy," in *Proc. USENIX Symp. Operating Syst. Design Implementation (OSDI)*, 2025, pp. 915–938.

[51] D. Kaviani, A. E. Ozdarendeli, J. Zhu, Y. Ding, and R. A. Popa, "Opal: Private memory for personal AI," arXiv:2604.02522, 2026.

[52] D. Rathee, J.-L. Watson, Z. N. Zhao, G. E. Suh, and R. A. Popa, "Onyx: Cost-efficient disk-oblivious ANN search," arXiv:2604.20401, 2026.

[53] M. Zhou, E. Shi, and G. Fanti, "Pacmann: Efficient private approximate nearest neighbor search," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.

[54] J. Li, Z. Huang, M. Zhang, J. Liu, C. Hong, T. Wei, and W. Chen, "Panther: Private approximate nearest neighbor search in the single server setting," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2025.

[55] J. Van Bulck, M. Minkin, O. Weisse, D. Genkin, B. Kasikci, F. Piessens, M. Silberstein, T. F. Wenisch, Y. Yarom, and R. Strackx, "Foreshadow: Extracting the keys to the Intel SGX kingdom with transient out-of-order execution," in *Proc. USENIX Secur. Symp.*, 2018.

[56] M. Anderson, G. Amit, and A. Goldsteen, "Is my data in your retrieval database? Membership inference attacks against retrieval augmented generation," arXiv:2405.20446, 2024.

[57] Z. Lin, C. Li, and K. Chen, "A survey on long-term memory security in LLM agents: Attacks, defenses, and governance across the memory lifecycle," *arXiv preprint arXiv:2604.16548*, 2026.

[58] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.

[59] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design," arXiv:2503.18813, 2025.

[60] J. Rehberger, "SpAIware: Uncovering a novel AI attack vector through persistent memory in LLM applications and agents," Embrace The Red, 2024, https://embracethered.com/blog/posts/2024/chatgpt-macos-app-persistent-data-exfiltration/.

[61] Unit 42, "When AI remembers too much: Persistent behaviors in agents' memory," Palo Alto Networks, Tech. Rep., 2025, https://unit42.paloaltonetworks.com/indirect-prompt-injection-poisons-ai-longterm-memory/.

[62] D. Das, J. Piet, D. Kaviani, L. Beurer-Kellner, F. Tramèr, and D. Wagner, "Trojan hippo: Weaponizing agent memory for data exfiltration," *arXiv preprint arXiv:2605.01970*, 2026.

[63] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, "Mem0: Building production-ready AI agents with scalable long-term memory," in *European Conference on Artificial Intelligence (ECAI)*, ser. Frontiers in Artificial Intelligence and Applications. IOS Press, 2025.

[64] D. Harnik, B. Pinkas, and A. Shulman-Peleg, "Side channels in cloud services: Deduplication in cloud storage," *IEEE Security & Privacy*, vol. 8, no. 6, pp. 40–47, 2010.

[65] M. Bellare, S. Keelveedhi, and T. Ristenpart, "Message-locked encryption and secure deduplication," in *Advances in Cryptology – EUROCRYPT 2013*, 2013, pp. 296–312.

[66] C. Gu, X. L. Li, R. Kuditipudi, P. Liang, and T. Hashimoto, "Auditing prompt caching in language model APIs," in *Proc. Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 267, 2025, pp. 20 477–20 496.

[67] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef, "Zep: A temporal knowledge graph architecture for agent memory," arXiv:2501.13956, 2025.

[68] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, D. Lian, and J.-Y. Nie, "C-Pack: Packed resources for general Chinese embeddings," in *Proc. Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval (SIGIR)*, 2024.

[69] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models," in *Proc. NeurIPS Datasets and Benchmarks Track*, 2021.